



CONTROL DE VERSIONES

Autor(es)	Fecha de modificación	Versión	Descripción del cambio	Revisó	Estado
VSSL	02/12/2024	1.0	Creación del Documento	IIB	Revisado
VSSL	09/12/2024	1.1	Modificación del Documento	----	Pendiente

Propósito

El manual de operaciones o manual técnico proporciona una guía para el correcto funcionamiento del proyecto desarrollado en Trabajo Terminal 2, abarcando la instalación, configuración y puesta en marcha de la aplicación móvil desarrollada para la observación de aves.

Esta herramienta permite a los usuarios capturar fotografías o seleccionar imágenes desde la galería, identificando automáticamente el tipo de ave y mostrando información relevante sobre esta, a partir de ello la aplicación permite realizar bitácoras y muestreos para una organización de cada sesión de observación de aves. Adicionalmente se desarrolló una pagina web donde se visualizan las bitácoras y muestreos realizados por los usuarios.

El documento está diseñado para apoyar a desarrolladores, técnicos y personal encargado de la implementación del sistema, describiendo los requerimientos de hardware y software, las restricciones del producto, y los pasos necesarios para el correcto funcionamiento del proyecto, incluyendo aplicación móvil y página web.

Este manual facilitará la replicación y puesta en marcha de las herramientas desarrolladas, asegurando que el producto final con su funcionamiento de manera eficaz y sin mayores complicaciones.



Actividades

Concepto	Descripción
Criterios de Entrada	● Requerimientos de hardware. Teléfonos móviles de gama media-alta en adelante <u>Procesador</u>: Snapdragon serie 700, Mediatek Dimensity serie 800/900, o equivalentes. Procesadores Octa-core con velocidades de hasta 2.4 GHz y tecnología de 7 nm o 6 nm para eficiencia y potencia. <u>Memoria RAM</u>: 4 a 8 GB. <u>Almacenamiento Interno</u>: 128 a 256 GB <u>Cámara</u>: Gran angular de 13 MP (f/1.8) con magnificación máxima de 0.21x, Ultra gran angular de 2 MP (f/2.2) con magnificación máxima de 0.21x, Lente de profundidad de 2 MP (f/2.4). Funciones de enfoque automático, flash LED, geoetiquetado, HDR. Lentes adicionales como ultra gran angular y lente de profundidad son recomendables, pero no esenciales. ● Requerimientos de software. <u>Memoria</u>: 79.9 MB para instalar la aplicación, mas espacio para almacenar imágenes. <u>Procesamiento</u>: 2.4 GHz de velocidad. <u>Memoria RAM</u>: 2 GB mínimo para el procesamiento de los modelos de identificación y clasificación. ● Sistema operativo compatible para su instalación. App Móvil: Android 10 o superior Página Web: Navegador actual ● Características y requisitos de los servidores de aplicaciones. App Móvil: Kotlin y Java <u>Dependencias</u>: TensorFlow Lite, Google Play Services Location, Github Glide, Room, Retrofit2 API de backend: Node.js, MongoDB <u>Dependencias</u>: Bcrypt, Bootstrap, Cors, Dotenv, EJS, Express, JSONWebToken, Mongoose, Morgan, PostageApp, Serve-favicon, Nodemon <u>Requisitos de servidor</u>: CPU con procesador multinúcleo, RAM de 2 GB mínimo, Almacenamiento mínimo de 71.6 MB ● Características y requisitos del servidor de base de datos Proveedor del servicio MongoDB Atlas (Base de datos como servicio DBaaS). Plan Gratis con Almacenamiento de 512 MB, RAM, vCPU compartida y OPS/SEC de 0 a 100



- **Navegadores compatibles.**

Para la funcionalidad web o el despliegue complementario:
Google Chrome (v91 o superior).
Safari (v14 o superior).
Microsoft Edge (v91 o superior).
Mozilla Firefox: Versión 89 o superior.

- **Protocolos de seguridad.**

HTTPS: transferencia segura de datos entre el cliente y los servidores.
Token JWT: encriptación de endpoints para usuario y administrador
Cors: administrar las solicitudes entre servidores.

- **Puertos de comunicación**

Puerto 10000 dado por el servicio de Render

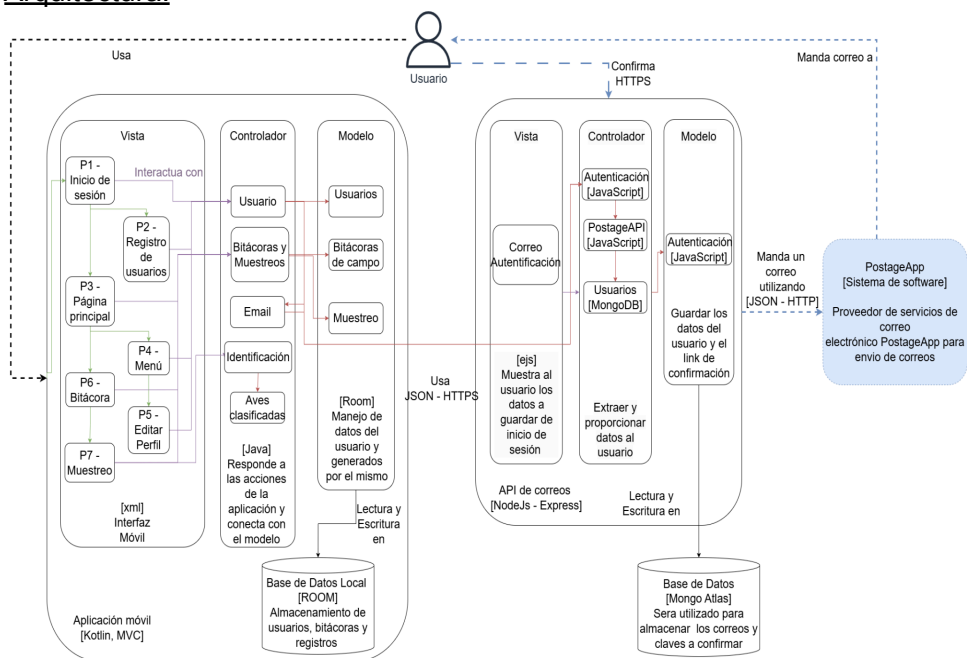
- **Interfaces con otros sistemas**

APIs externas: PostageApp para el envío de correos
Base de datos: MongoDB Atlas para el almacenamiento
Modelos de IA: Yolo v8 s y MobileNet v3 large, redes neuronales entrenadas para identificación y clasificación de imágenes de aves.

Framework
y
estándares

Aplicación Móvil

Arquitectura:





La aplicación sigue el patrón Modelo-Vista-Controlador (MVC), el cual divide la lógica de negocio, la presentación y la interacción con los usuarios en capas separadas.

Componentes Principales

1. Vista (Interfaz Móvil) [XML]:

La vista representa la interfaz de usuario desarrollada con XML y Kotlin. Aquí es donde el usuario interactúa con la aplicación. Pantallas principales:

- P1: Inicio de sesión.
- P2: Registro de usuarios.
- P3: Página principal.
- P4: Menú.
- P5: Editar perfil.
- P6: Bitácoras.
- P7: Muestreo.

2. Controlador:

El controlador recibe las acciones de la interfaz, interactúa con el modelo (datos locales o API) y actualiza la vista. Componentes importantes:

- Usuario: Maneja las acciones relacionadas con el inicio de sesión, registro y perfil del usuario.
- Bitácoras y Muestreos: Controla la creación, edición y sincronización de las bitácoras y registros.
- Identificación de Aves: Utiliza el modelo local TensorFlow Lite para identificar y clasificar aves.

3. Modelo:

- Usuarios: Representa la información del usuario.
- Bitácoras de campo: Almacena registros creados por el usuario.
- Muestreo: Registros específicos dentro de una bitácora.
- Base de Datos Local (Room): Persistencia de datos locales para bitácoras y registros sin conexión a internet.

4. API de correos:

- Comunicación con el backend mediante JSON-HTTP para sincronizar los datos locales (bitácoras, usuarios y muestreos) con los de la nube y enviar correos a través de PostageApp.

Flujo de Datos

- Inicio de sesión y registro:
 1. El usuario ingresa los datos en las pantallas P1 o P2.
 2. El controlador envía una solicitud HTTP a la API de autenticación.
 3. El backend valida los datos y envía un token JWT en caso de éxito.
 4. El usuario confirma el correo enviado con el token asignado.
 5. La aplicación almacena el token localmente y redirige a P3 (Página Principal).



- Sincronización de bitácoras:
 1. El usuario registra una bitácora en P6 (Bitácoras).
 2. La información se guarda en la base de datos local usando Room.
 3. Al sincronizar, el controlador envía los datos al backend utilizando la API.
 4. El backend guarda los registros en MongoDB Atlas.
- Identificación de aves:
 1. El usuario toma una foto o selecciona una imagen.
 2. El controlador identifica si existe una imagen con el modelo entrenado de YOLOv8 s.
 3. El controlador procesa la imagen y la clasifica usando el modelo entrenado de MobileNetV3.
 4. Se muestra el nombre y características del ave en la pantalla P6

Funciones críticas:

Identificación de aves: Modelo de YOLOv8 s y MobileNetV3 con funciones como `classifyBird(image: Bitmap)`, `detectBird(image: Bitmap)`: Boolean

Gestión de bitácoras: Sincronización con la base de datos de la nube como `subirBitacoraANube( bitacora: Bitacora, context: Context)`, `sincronizarDatos()`: Boolean

Manejo de datos locales: Persistencia con Room para bitácoras y datos temporales como `guardarMuestreo(userId: String?)`, `guardarBitacora(userId: String?)`, `registrarUsuarioLocal()`

Lenguajes de programación: Kotlin y Java por su compatibilidad nativa con Android.

Frameworks y dependencias:

TensorFlow Lite (v2.14.0): Implementación de modelos de Yolo y MobileNet para la identificación de aves.

Google Play Services Location (v21.0.1): Para la obtención precisa de datos de ubicación por coordenadas.

Glide (v4.15.1): Gestión eficiente de carga y visualización de imágenes.

Room (v2.6.1): Framework de persistencia para manejo de base de datos local SQLite.

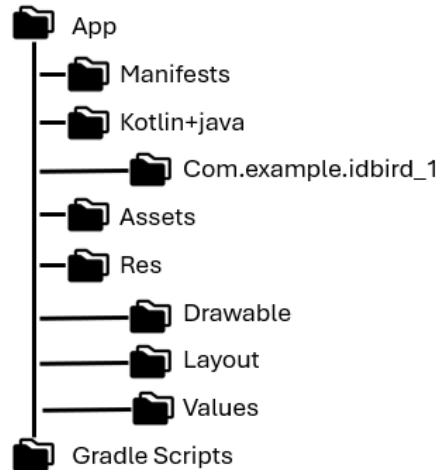
Retrofit2 (v2.9.0): Cliente HTTP para comunicación con APIs REST.

Diseño del sistema

Estructura del código:



Aplicación Móvil



Carpetas:

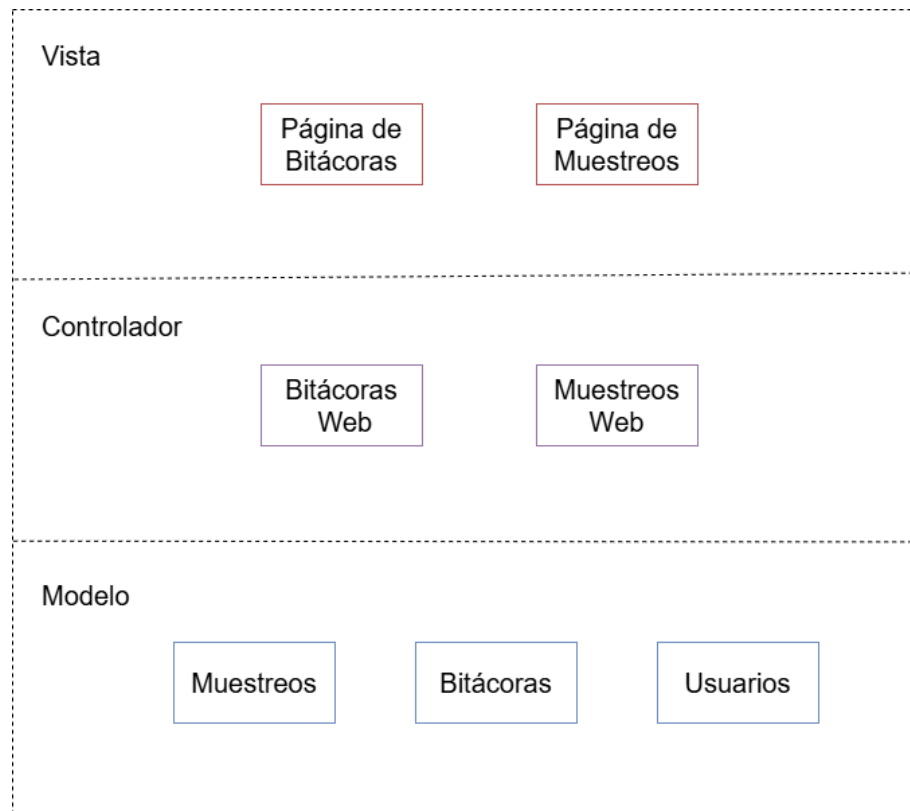
- **App:** La carpeta principal del proyecto donde se encuentra todo el código fuente y recursos de la aplicación
- **Manifests:** Contiene el archivo `AndroidManifest.xml`, el cual describe información esencial sobre la aplicación, como, componentes principales, actividades, permisos necesarios y configuración de la aplicación.
- **Kotlin+Java:** Contiene el código fuente de la aplicación organizado dentro del paquete `com.example.idbird_1`, usando los lenguajes de Kotlin y Java para complementar su funcionamiento
- **Com.example.idbird_1:** Contiene las actividades de cada pantalla, las clases para el uso de la base de datos con sus modelos, DAOs y adapters necesarios, así como componentes de servicios para conexión a API.
- **Assets:** Carpeta que contiene recursos adicionales como los archivos de modelos entrenados de Yolo y MobileNet.
- **Res:** Contiene recursos visuales y de diseño utilizados por la aplicación, los indispensables en esta aplicación son `drawable`, `layout` y `values`.
- **Drawable:** Carpeta donde se almacenan imágenes, íconos y otros recursos gráficos utilizados en la aplicación.
- **Layout:** Contiene archivos XML que definen el diseño de las interfaces de usuario de cada pantalla.
- **Values:** Contiene archivos XML para definir el estilo de la aplicación y valores reutilizables, como cadenas, colores y formatos.



API de Backend:

Arquitectura:

El backend sigue el patrón Modelo-Vista-Controlador (MVC) adaptado a servicios web, utilizando Node.js y Express:



Componentes Principales

1. Vista:

Utiliza EJS (Embedded JavaScript) para renderizar las vistas con datos dinámicos. Páginas principales:

- Página de Bitácoras: Muestra las bitácoras sincronizadas.
- Página de Muestreos: Visualización de los muestreos específicos de cada bitácora.

2. Controlador:

- Bitácoras Web: Gestiona las solicitudes relacionadas con la visualización y búsqueda de bitácoras.
- Muestreos Web: Controla la consulta y visualización de los muestreos específicos.

3. Modelo:

Almacenamiento de usuarios, bitácoras y registros sincronizados desde la aplicación móvil en MongoDB Atlas.



- Usuarios: Información del usuario autenticado.
- Bitácoras: Datos sincronizados desde la aplicación móvil.
- Muestreos: Registros relacionados con las bitácoras.

Flujo de Datos

- Visualización de Bitácoras y Muestreos:
 1. La vista realiza una solicitud al controlador Bitácoras Web.
 2. El controlador consulta la base de datos (MongoDB Atlas) y devuelve las bitácoras sincronizadas.
 3. Los datos se renderizan dinámicamente en la página usando EJS.

Funciones críticas:

Autenticación de usuario: Registro, validación y restricciones para usuarios con JSON Web Tokens, como authentication.middleware.js

Envío de Correos: Utiliza PostageApp para confirmaciones de registro.

Manejo de datos en la nube: Creación, actualización y consultas a la base de datos en la nube en MongoDB Atlas, para usuarios, bitácoras, muestreos y aves.

Framework principal: Node.js (v20.17.0), utilizando JavaScript como lenguaje de programación.

Base de datos: MongoDB gestionada en la nube mediante MongoDB Atlas.

Dependencias del Backend:

Bcrypt(v5.1.1): Manejo de hashing para tokens de usuarios.

Bootstrap (v5.3.3): Librería CSS para interfaces web.

CORS(v2.8.5): Configuración para control de acceso desde diferentes orígenes.

Dotenv(v16.4.5): Gestión de variables de entorno.

EJS (Embedded JavaScript) (v3.1.10): Motor de plantillas para renderización de vistas en el backend.

Express (v4.19.2): Framework para construir el servidor y definir rutas.

JSONWebToken (JWT) (v9.0.2): Manejo de autenticación segura mediante tokens.

Mongoose (v8.6.0): ODM para interactuar con MongoDB.

Morgan (v1.10.0): Middleware para registro de peticiones HTTP.

PostageApp (v2.0.0): Para el manejo de notificaciones o correos electrónicos.

Serve-favicon (v2.5.0): Middleware para servir iconos desde el backend.

Nodemon (v3.1.7): Herramienta para el desarrollo que reinicia el servidor automáticamente al detectar cambios en el código.

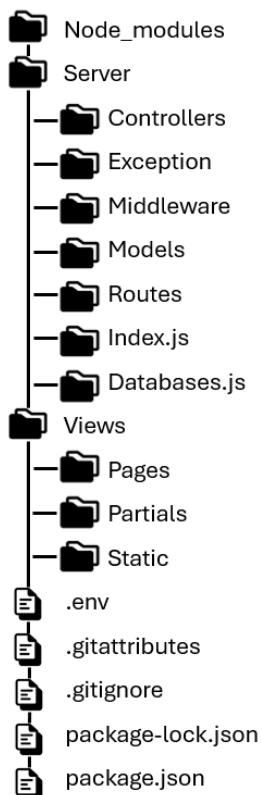


Diseño del sistema

Estructura del código:

La organización del código en el repositorio sigue una estructura modular y jerárquica que facilita la comprensión, el mantenimiento y la escalabilidad del sistema.

API de Correos



Carpetas:

- **node_modules:** Carpeta generada automáticamente por npm. Contiene las dependencias y librerías necesarias para que la aplicación funcione. Se excluye del control de versiones con `.gitignore` porque puede ser reconstruida con el archivo `package.json`.
- **Server:** Contiene la lógica principal del servidor. Se divide en submódulos para cumplir con la arquitectura Modelo-Vista-Controlador (MVC).
- **Controllers:** Contiene los controladores que manejan la lógica de negocio y la interacción entre las rutas y los modelos. Como el controlador de usuarios, correos, bitácoras, muestreos y aves.
- **Exception:** Contiene manejadores de errores personalizados. Contiene los códigos de errores para cada caso necesario, además de las excepciones para errores de base de datos, errores HTTP y



	las excepciones estándar. ● Middleware: Contiene middlewares que interceptan solicitudes HTTP. Utiliza componentes para verificar las consultas y respuestas de cada solicitud con un middleware de autenticación y de errores. ● Models: Define las estructuras de datos y la interacción con la base de datos, para las tablas de aves, bitácoras, muestreos y usuarios. ● Routes: Define las rutas de la API y sus endpoints, usando rutas para las consultas de aves, bitácoras, muestreos, correos, usuarios y para manejar la página web. ● Views: Contiene las vistas dinámicas que se renderizan en el lado del servidor utilizando EJS. ● Pages: Contiene páginas completas que se envían al usuario, como la página de error, la de confirmación de correo y las dos de la página web para visualizar las bitácoras y los muestreos. ● Partials: Contiene fragmentos reutilizables, como los encabezados. ● Static: Contiene archivos estáticos como imágenes, logos o estilos de CSS. Archivos raíz: ● .env: Almacena variables de entorno como claves secretas y URLs. ● .gitattributes y .gitignore: Configuran reglas para el control de versiones, en especial para definir los archivos a ignorar. ● package.json y package-lock.json: Define las dependencias y metadatos del proyecto. ● index.js: Archivo principal que inicializa el servidor, define middlewares globales y conecta las rutas. ● databases.js: Configura la conexión con la base de datos (ej. MongoDB Atlas).
Restricciones del producto	● Permisos de ejecución del producto. Permisos en el dispositivo móvil para acceder a cámara, galería de fotos y la ubicación GPS ● Las políticas de uso para cada usuario. Solo se permite una cuenta por usuario registrada con un correo electrónico válido. Los usuarios deben aceptar los términos y condiciones, incluyendo el manejo de datos personales (como ubicación e imágenes). La aplicación está destinada exclusivamente para fines de observación de aves y aprendizaje. ● Restricciones de recursos. Los modelos de redes neuronales para identificación de aves están optimizados para dispositivos con procesadores de gama media-alta; en



	dispositivos de menor rendimiento, podría haber demoras en el procesamiento de imágenes. El almacenamiento de datos local está limitado por el espacio disponible en el dispositivo del usuario. La funcionalidad de identificación depende de la conectividad a internet para acceder a las API externas y la base de datos en la nube. ● Restricciones de tiempo. La identificación automática puede tardar entre 1 y 5 segundos dependiendo de la calidad de la imagen. La descarga de bitácoras guardadas en la nube puede variar entre 1 y 10 segundos dependiendo de la cantidad de bitácoras guardadas por el usuario y la calidad del internet. El registro de usuario, primer inicio de sesión, sincronización de datos, guardado en nube y descarga de bitácoras se pueden realizar solo con conexión a internet. ● Restricciones de alcance. La base de datos inicial está limitada a la información de 5 especies de aves cuasi endémicas de Zacatecas. Es posible modificar y aumentar la información o las aves desde la base de datos de MongoDB. La precisión de la identificación se limita a las 5 aves incluidas en el modelo entrenado. No se reconocerán aves no entrenadas. ● Restricciones de calidad. Las imágenes borrosas, de baja calidad o con iluminación deficiente pueden reducir la precisión de la identificación. La funcionalidad de ubicación GPS depende de la precisión del sensor del dispositivo y la disponibilidad de una buena conexión satelital.
Obtener acceso y/o descarga al producto	Descarga de aplicación: https://drive.google.com/drive/folders/1yzZfqxg4xhHlkc5vjk6_qKG4tMvqBvH2?usp=sharing Repositorio privado de la API – requieren permisos VaniaLee10/API-de-correos Repositorio privado de la App – requieren permisos VaniaLee10/App_IdBird
Proceso de instalación	● Instalación y configuración del producto. ●

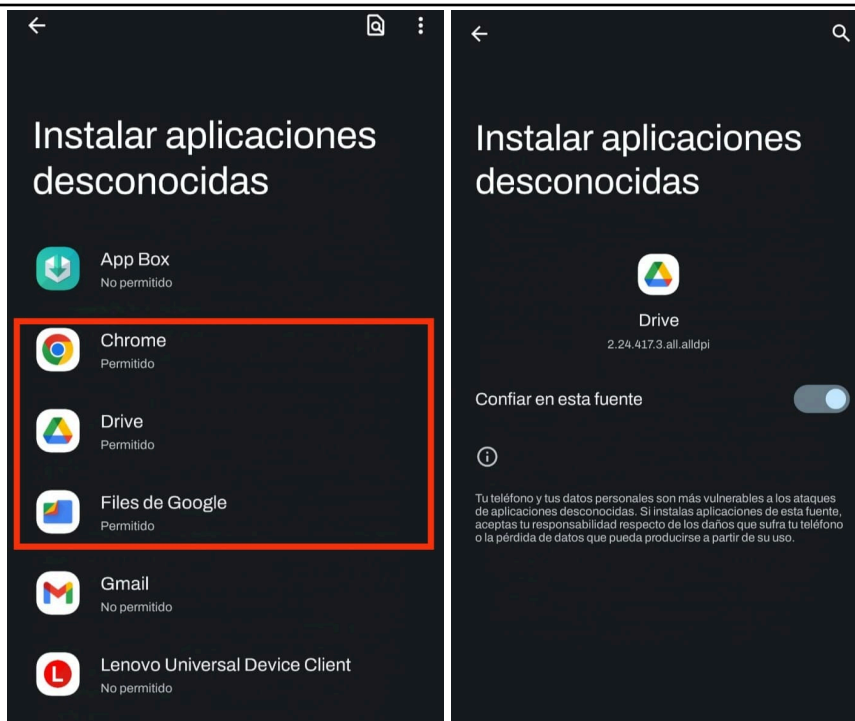


y
configuración
n del
producto.

Aplicación Móvil:

1. Asegúrate que tu celular permita instalar apps desconocidas
 - a. Ir a “Ajustes”
 - b. Buscar la sección de “Aplicaciones”
 - c. Buscar el apartado de “Instalar apps desconocidas”
 - d. Activar la opción “Permitir desde esta fuente” o buscar Drive o Chrome y activar la opción “Confiar en esta fuente”





2. Descargar la aplicación desde el link de Drive
3. Abre la aplicación y regístrate o inicia sesión con una cuenta, asegurándote de tener internet para estos pasos.

- Instalación y configuración de los servidores de aplicaciones.

API:

1. Obtener acceso al repositorio de GitHub y clonarlo desde https://github.com/VaniaLee10/App_IdBird
2. Instalar dependencias del proyecto, con "npm install", asegurate de tener Node.js v20.17.0

```
PS C:\Users\Vania\OneDrive\Documentos\TT\API de correos> npm install

up to date, audited 216 packages in 2s

26 packages are looking for funding
  run `npm fund` for details

7 vulnerabilities (3 low, 4 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```



3. Crea un archivo de .env con las siguiente variables, Asegúrate de no compartir ni subir el archivo .env al repositorio. El archivo debe estar en el .gitignore.

```
DB_URI='mongodb+srv://Vania_10:Vania_10@idbird.psrme.mongodb.net/?retryWrites=true&w=majority&appName=IdBird'
TOKEN_KEY=VASF_1&0
AUTHENTICATION_KEY=1D_81RD_4P1
URL_POSTAGEAPP = 'https://api.postageapp.com/v.1.0/send_message.json'
URL_CORREO_POSTAGEAPP = 'https://idbird-api.onrender.com/user/confirmado/'
TOKEN_POSTAGEAPP = 'TC14BUIDeXTUTICRXleLTaMEY45yJHv'
```

```
API de correos
├── node_modules
├── server
├── views
├── .env
├── .gitattributes
├── .gitignore
├── API_Endpoints_Doc.docx
├── package-lock.json
└── package.json
```

4. Ejecutar el servidor con “npm run dev”

```
PS C:\Users\Vania\OneDrive\Documentos\TT\API de correos> npm run dev

> api-de-correos@1.0.0 dev
> nodemon server/index.js

[nodemon] 3.1.7
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server/index.js`
Server en puerto 3000
DB is connected
```

5. Subir cambios al repositorio de GitHub
6. Ingresar a Render de [IdBird-API](#) · [Web Service](#) · [Render Dashboard](#) y asegurarse que el servicio web esté activo.

- Instalación y configuración del servidor de base de datos

Base de Datos:

1. Ingresar a la base de datos de MongoDB Atlas [Project Overview | Cloud: MongoDB Cloud](#). En caso de requerir una base de datos



nueva:

- Crear un cluster desde su cuenta de MongoDB Atlas.
- En el archivo .env cambiar la variable DB_URI por la url de su base de datos
- Cree las tablas “aves”, “bitácoras”, “muestreos” y “usuarios”
- Carga inicial de datos en la tabla de “aves”:

```
{"_id":{"_id":"672ba8645fea1767abbe20ec"},"nombre_coloquial":
"Carbonero Mexicano","color":"Negro, Blanco,
Gris","dimensiones":"12-13 cm x
20cm","nivel_endemismo":"Cuasiendémicas","actualizado":true,"
nivel_extincion":"Bajo"}
{"_id":{"_id":"672ba8925fea1767abbe20ec"},"nombre_coloquial":
"Carpintero de Arizona","color":"Rojo, Cafe, Blanco, Negro,
Gris","dimensiones":"20 cm x 33
cm","nivel_endemismo":"Endemica","actualizado":true,"nivel_ex
tincion":"Bajo"}
{"_id":{"_id":"672ba8c95fea1767abbfa71c"},"nombre_coloquial":
"Papamoscas Cardenalito","color":"Rojo, Naranja,
Negro","dimensiones":"15 cm x 25
cm","nivel_endemismo":"Endemica","actualizado":true,"nivel_ex
tincion":"Bajo"}
{"_id":{"_id":"672ba8f35fea1767abc041e8"},"nombre_coloquial":
"Papamoscas Pinero","color":"Gris, Verde","dimensiones":"13
cm x 20
cm","nivel_endemismo":"Cuasiendémicas","actualizado":true,"ni
vel_extincion":"Bajo"}
{"_id":{"_id":"672ba9205fea1767abc10043"},"nombre_coloquial":
"Zacatonero Serrano","color":"Café canela, Negro, gris,
blanco","dimensiones":"17 cm x 25
cm","nivel_endemismo":"Endemica","actualizado":true,"nivel_ex
tincion":"Bajo"}
```

- Configuración de las Interfaces con Otros Sistemas:

PostageApp:

PostageApp es un servicio externo utilizado para el envío de correos electrónicos de confirmación durante el registro de usuarios. La API permite enviar correos utilizando plantillas predefinidas con datos dinámicos como nombre del usuario, correo y enlace de confirmación. Se consume a través de solicitudes HTTP POST con datos en formato JSON. El servidor backend envía los datos del usuario y utiliza la clave de acceso de la API para autenticar la solicitud.

Pasos para su implementación:

- Ingresa al proyecto de PostageApp en [PostageApp - BirdVision - IdBird - Message Details](#). En caso de requerir una cuenta nueva:
 - Crear una cuenta en PostageApp, puede ingresar sin un plan de pago si no lo requiere.
 - Crear un proyecto llamado “IdBird”



c. Agregar un correo para envío de correos y validarlo

IdBird



[EDIT PROJECT](#) [REMOVE PROJECT](#) [SHOW HELP](#)

Hour

Today

Week

Month

METRIC	PERCENTAGE	CURRENT	PREVIOUS
Transmissions Created		0	6
Queued to Send		0	0
Delivered	0%	0	5
Undeliverable	0%	0	0
Bounced	0%	0	1
Spammed	0%	0	0
STATISTICS FOR THE 5 MESSAGES THAT WERE DELIVERED:			
0.0% were opened		0	0

d. Crear dos plantillas

IdBird_Layout

```
<div class = 'container'>
  <div class = 'header'>
    
    <h1>IdBird </h1></div>
    <div class = 'content'> {{ * }} <h5> Atentamente,
    IdBird </h5> <br>
    <p class = 'note'>En caso de no haberse registrado ignore
    este correo.</p>
  </div>
</div>
```

IdBird_child Layout: IdBird_layout

```
<p>Gracias por tu registro. {{nombre}}</p>
<p>Haga click en el siguiente link para confirmar y completar
el registro</p>
<p>Correo registrado: {{correo}}</p>
<p>Contraseña registrada: {{contrasena}}</p>
<p>{{link}}</p>
<p><a href= "{{link}}" >Confirmar registro</a></p>
```




Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria de Ingeniería
Campus Zacatecas

Manual Técnico / Operaciones



SENDER: idbird@postageapp.com ✓ VALIDATED
RECIPIENT: 100 idbird.upiiz@gmail.com
FROM: idbird.upiiz@outlook.com ✗ NOT VALIDATED
SUBJECT: Registro en IdBird
TO: idbird.upiiz@gmail.com
AUTO-SUBMITTED: auto-generated

HTML PLAIN TEXT RAW SOURCE



IdBird

Gracias por tu registro. IdBird Upiiz

Haga click en el siguiente link para confirmar y completar el registro

Correo registrado: idbird.upiiz@gmail.com

Contraseña registrada: IdBird

https://idbird-api.onrender.com/user/confirmado/eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEwMTIwODAxIiwia25y8FrRC_bCIP6bXPGeOSGhFE

[Confirmar registro](#)

Atentamente, IdBird

e. En el archivo .env cambiar las variables URL_POSTAGEAPP, URL_CORREO_POSTAGEAPP, TOKEN_POSTAGEAPP por las dadas en la cuenta

Endpoint Utilizado:

URL: https://api.postageapp.com/v.1.0/send_message.json

Método: POST

Ejemplo de la solicitud enviada:

```
var options = {
  recipients: destinatario,
  headers: {
    subject: "Registro en IdBird",
    from: "idbird.upiiz@outlook.com",
  },
  template: "IdBird_child",
  variables: {
    aplicacion: "IdBird",
    nombre: nombre_destinatario,
    link: link,
    correo: usuario.correo,
    contrasena: usuario.contrasena,
  },
};
```

Ejemplo de la respuesta recibida:

```
"respuesta_correo":{
  "status": true,
  "respuesta":{
    "message":{
```



```
"id": 110079983,  
"uid": "0dbd8086-663f-4c86-afe3-9aef537e5136",  
"url":  
"https://cimat-unidad-zacatecas.postageapp.com/projects/60525  
/messages/110079983",  
"priority": "transactional",  
"status": "queued"}  
},  
"link":  
"https://idbird-api.onrender.com/confirmado/eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEwNzI1MTAyMDI0MjEzMjAyIiwiaWF0IjoxNzI5ODkxOTQwLCJleHAiOjE3Mjk5MjA3NDB9.P6_5XGC6yKT8Tq7lVFL3CRaeISR4tm9gEEZyKdQ0e1E",  
"token":  
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEwNzI1MTAyMDI0MjEzMjAyIiwiaWF0IjoxNzI5ODkxOTQwLCJleHAiOjE3Mjk5MjA3NDB9.P6_5XGC6yKT8Tq7lVFL3CRaeISR4tm9gEEZyKdQ0e1E"  
}
```

Implementación en el backend, código en Node.js:

```
correoCtrl.enviarCorreo = async (usuario, token) => {  
  id_usuario = usuario.id_usuario;  
  if (!id_usuario) { return new StandarException('ID de  
usuario no definido', codigos.validacionIncorrecta); }  
  const link =  
`https://idbird-api.onrender.com/user/confirmado/${token}`;  
  var postageapp = new  
PostageApp("TC14BUIDeXTUTICXRleLTaMEY45yJHv");  
  const destinatario = usuario.correo;  
  if (!destinatario || !esCorreoValido(destinatario)) {  
return new StandarException('Correo no definido o formato  
incorrecto', codigos.validacionIncorrecta); }  
  let nombre_destinatario;  
  if (!usuario.nombre) { nombre_destinatario =  
usuario.correo;}  
  else {  
    nombre_destinatario = usuario.nombre;  
    if (usuario.apellido) {nombre_destinatario += " " +  
usuario.apellido;}  
  }  
  var options = {  
    recipients: destinatario,  
    headers: {  
      subject: "Registro en IdBird",  
      from: "idbird.upiiz@outlook.com",  
    },  
    template: "IdBird_child",  
    variables: {  
      aplicacion: "IdBird",  
      nombre: nombre_destinatario,  
      link: link,
```



	<pre>correo: usuario.correo, contrasena: usuario.contrasena, }, }; if (!options.recipients !options.headers.subject !options.headers.from !options.template !options.variables.link) { return new StandarException('Estructura del correo no válida', codigos.validacionIncorrecta); } const response = await postageapp.sendMessage(options).catch(error => null); if (response === null) { return new StandarException('Error al enviar el correo', codigos.errorAlEnviarCorreo); } return{ status: true, respuesta: response, link: link, token: token }; };</pre> • Pruebas y depuración: Para garantizar el correcto funcionamiento del sistema y la calidad del producto, se llevaron a cabo pruebas unitarias, de integración y de sistema. Se recomienda realizar cada una de estas pruebas cuando se realiza un cambio nuevo antes de aprobarlo en producción. Se recomienda usar métodos de depuración en el IDE usado, se recomienda usar Android Studio para la aplicación y Visual Studio Code para la API, para las pruebas unitarias. Para las pruebas de integración y de sistema se sugiere usar herramientas como Postman y Talend API Tester para comprobar el correcto funcionamiento de las rutas y sus funciones.
Despliegue del producto	<u>Servicios Utilizados</u> Aplicación móvil: Drive para su distribución, Android Studio para su desarrollo API: Render para su despliegue, GitHub para su almacenamiento, Visual Studio para su desarrollo Base de datos: MongoDB Atlas para su almacenamiento y gestión en la nube APIs Externas: PostageApp para el envío de correos <u>Despliegue</u> 1. Activar el servicio web de Render



2. Abrir la página web o la aplicación

La aplicación puede ser usada sin conexión a internet o al servidor en línea.

Seguridad:

En el desarrollo del proyecto se integraron diversas prácticas de seguridad para proteger la información de los usuarios y garantizar la integridad de los datos desde la aplicación y la API.

Se usa la autenticación mediante tokens para garantizar que solo usuarios autenticados puedan acceder a las funcionalidades protegidas del sistema, se implementó el estándar JWT (JSON Web Token) y un middleware para comprobar la autenticación en las consultas.

Se genera un token cuando un usuario se registra y solo con este mismo token se puede confirmar el correo de autenticación.

Además, se limita el acceso a rutas solo de desarrollador con tokens en el encabezado de la solicitud.

Los tokens necesarios se deben encontrar en el archivo .env del proyecto para su uso y modificación en caso de ser necesario.

Documentación de Código:

Documentación de Endpoints:

Archivo específico donde se documentan todos los endpoints disponibles en la API del backend. Este archivo incluye las rutas disponibles, métodos HTTP utilizados, parámetros requeridos y opcionales, ejemplos de solicitudes y respuestas.

Ubicación: El archivo se encuentra en el repositorio del backend, bajo el nombre API_Endpoints_Doc.docx

Comentarios en el código:

Los comentarios deben ser claros, concisos y seguir las mejores prácticas. Se utilizan para explicar la funcionalidad de métodos críticos y bloques de lógica compleja.

Para garantizar la claridad y consistencia en la documentación del código fuente, se utiliza el estándar JSDoc en el backend y un formato equivalente en otros lenguajes. Este estándar permite describir funciones, parámetros, y retornos de forma estructurada:

```
/**
 * @description [Breve descripción de lo que hace la función]
 * @param {[Tipo]} [Nombre] - [Descripción del parámetro]
 * @returns {[Tipo]} [Descripción del valor retornado]
 */
```

Control de Versiones:

Se utiliza Git como sistema de control de versiones, alojado en



Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria de Ingeniería
Campus Zacatecas

Manual Técnico / Operaciones



	repositorios privados en GitHub para el código de la aplicación y por otro lado para el código de la API. Estrategias de ramas: Cada cambio realizado se debe generar en una nueva rama con nomenclatura “feature_InicialesAutor_Cambio” Cuando sean aprobados y hayan pasado las pruebas respectivas es posible integrarlo en la rama “Main”. Estrategias de cambios: Cada cambio en el código debe registrarse mediante un commit con un mensaje claro y descriptivo.
--	--